Experiment 8

Aim: Implement Principal Component Analysis technique.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv("iris.csv")

print(df.head())
print("Shape:", df.shape)
print(df.info())
```

```
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
Shape: (150, 5)
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    str
dtypes: float64(4), str(1)
memory usage: 7.2 KB
None
```

In [2]:
```
X = df.drop(columns=["species"])
y = df["species"]

print(X.head())
print("X shape:", X.shape)
```

```
   sepal_length  sepal_width  petal_length  petal_width
0           5.1          3.5           1.4          0.2
1           4.9          3.0           1.4          0.2
2           4.7          3.2           1.3          0.2
3           4.6          3.1           1.5          0.2
4           5.0          3.6           1.4          0.2
X shape: (150, 4)
```

In [3]:
```
import seaborn as sns

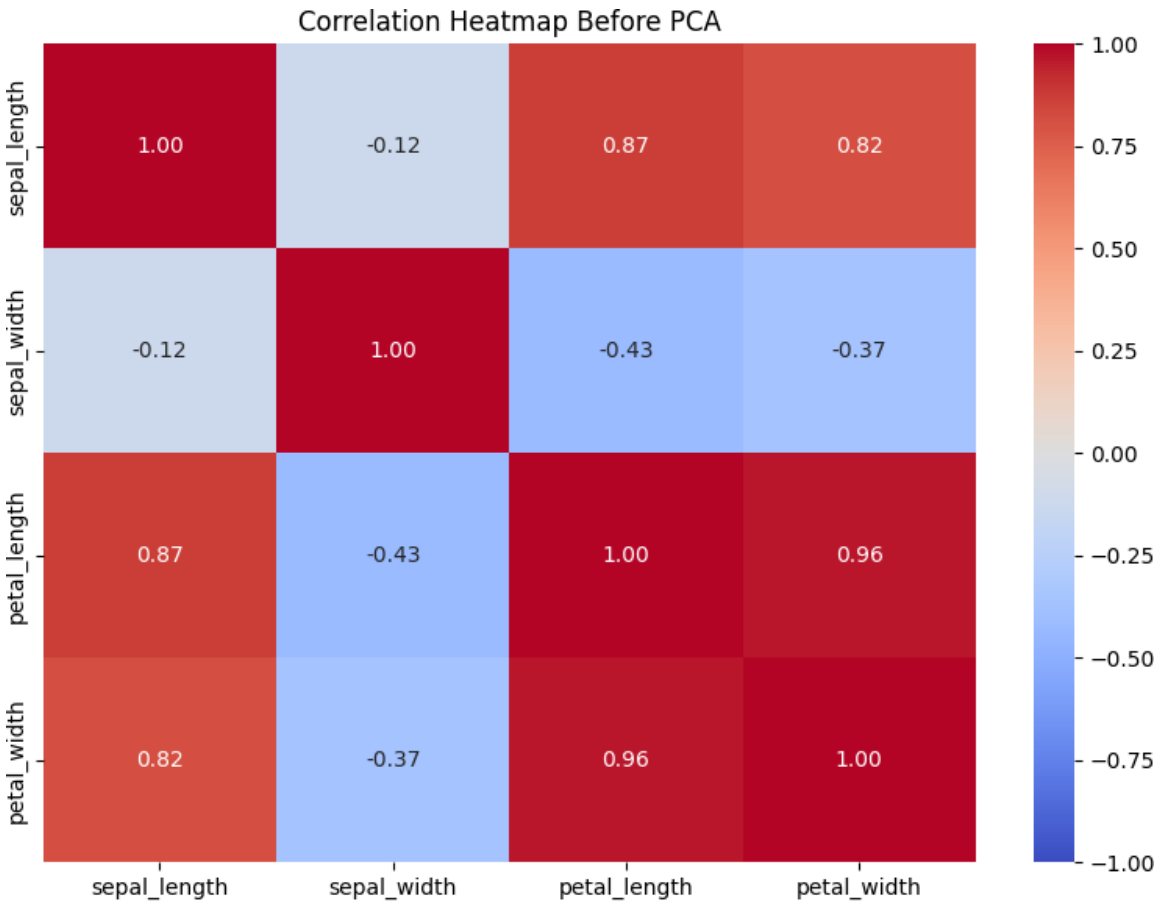
# Correlation heatmap before PCA
plt.figure(figsize=(8, 6))

correlation_matrix = X.corr()

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
```

```
)

plt.title("Correlation Heatmap Before PCA")
plt.tight_layout()
plt.show()
```



In [4]:
```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Original shape:", X.shape)
print("Scaled shape:", X_scaled.shape)
print(X_scaled[:5])
```

```
Original shape: (150, 4)
Scaled shape: (150, 4)
[[-0.90068117  1.01900435 -1.34022653 -1.3154443 ]
 [-1.14301691 -0.13197948 -1.34022653 -1.3154443 ]
 [-1.38535265  0.32841405 -1.39706395 -1.3154443 ]
 [-1.50652052  0.09821729 -1.2833891  -1.3154443 ]
 [-1.02184904  1.24920112 -1.34022653 -1.3154443 ]]
```

In [5]:
```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)

print("Original shape:", X_scaled.shape)
print("Reduced shape:", X_pca.shape)

print("Explained variance ratio:")
```

```
print(pca.explained_variance_ratio_)

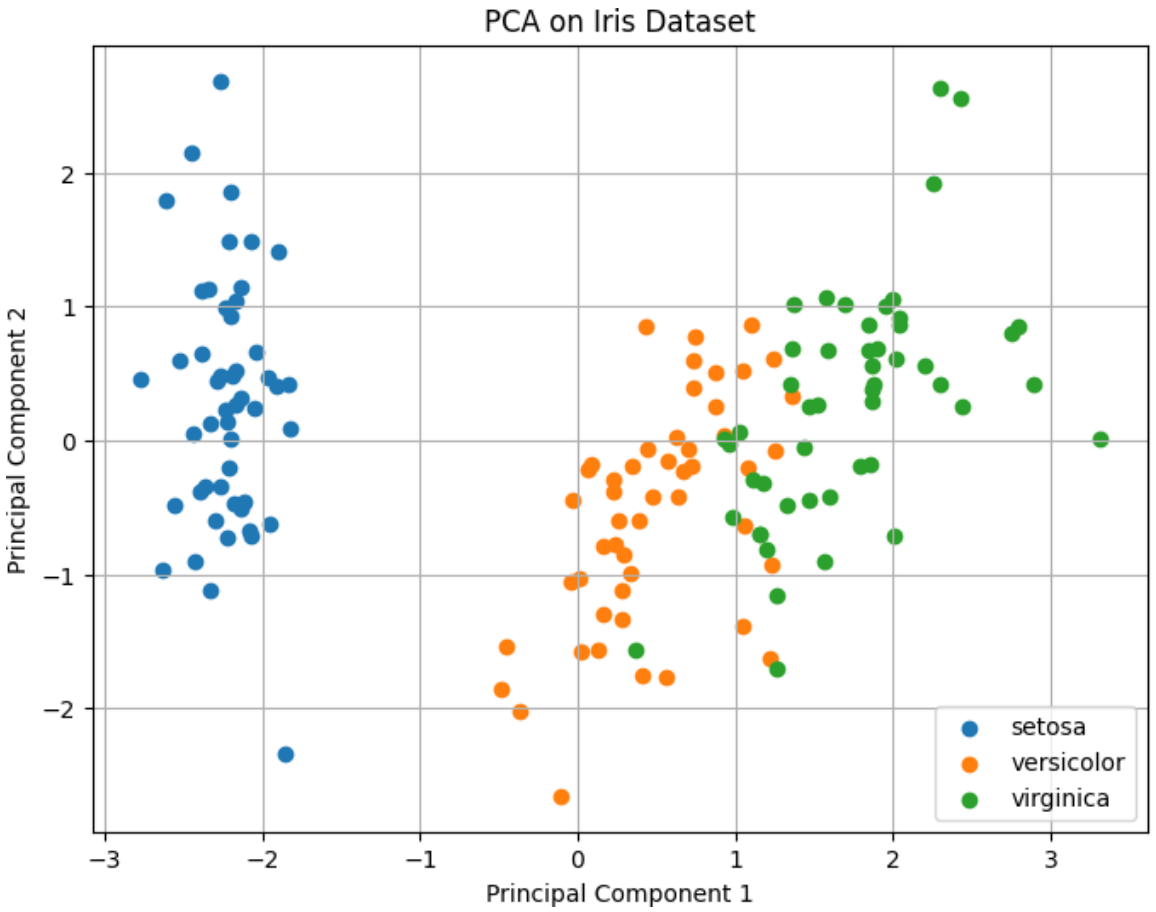
print("Total variance retained:")
print(pca.explained_variance_ratio_.sum())
```

```
Original shape: (150, 4)
Reduced shape: (150, 2)
Explained variance ratio:
[0.72962445 0.22850762]
Total variance retained:
0.9581320720000165
```

In [6]:
```
plt.figure(figsize=(8, 6))

for species in y.unique():
    mask = (y == species)
    plt.scatter(
        X_pca[mask, 0],
        X_pca[mask, 1],
        label=species
    )

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA on Iris Dataset")
plt.legend()
plt.grid(True)
plt.show()
```



In [ ]:
```

```